

<code>`struct ... : View`</code>	<code>`body`</code>
<code>`some View`</code>	<code>`@Environment`</code>
<code>`@Query`</code>	key path (<code>`\Type.prop`</code>)
<code>`NavigationStack`</code>	<code>`Group`</code>

The required property on a View -- what actually appears on screen.

Example: SwiftUI calls body again every time watched state changes, so keep expensive work out of it.

Every SwiftUI screen is a struct conforming to the View protocol.

Example: struct SettingsView: View is the starting shape of every screen you write.

Reads values handed down by the system or a parent view, like dismiss or modelContext.

Example: @Environment(\.dismiss) var dismiss gives a sheet the ability to close itself.

Returns a View, but the exact type stays unnamed -- the compiler works it out.

Example: Lets body return a deeply nested stack without you spelling out its real type.

A reference that points at a property rather than its value -- used in sorting and predicates.

Example: sort: \Item.purchaseDate tells a query which property to order by.

Fetches database records and keeps watching them -- the view updates when the data changes.

Example: @Query var items: [Item] repopulates the list automatically after a save, with no manual refetch.

An invisible container -- it draws nothing, but lets you apply one modifier to several alternatives.

Example: Wrap an if/else in Group so both branches get the same .toolbar without repeating it.

The container that provides a navigation bar, a title, and a toolbar.

Example: Wrapping a list in NavigationStack is what makes a title and Done button possible.